# От запроса до покупки

Разбираем, как устроен магазин.
Затем собираем его по шагам.

Понять → Собрать → Проверить




↳ Начнём с одной покупки и проследим, что делает каждый участник.

# Одна покупка — несколько действий

Аня уже вошла в аккаунт. Теперь она выбирает и покупает кроссовки.

01
**Каталог**
Найти North Runner

02
**Размер**
Выбрать 42

03
**Корзина**
Добавить 2 пары

04
**Заказ**
Подтвердить покупку

05
**История**
Увидеть свой заказ

**Каждая стрелка — новое действие**

Посмотреть товар, добавить позицию и оформить заказ — разные запросы.

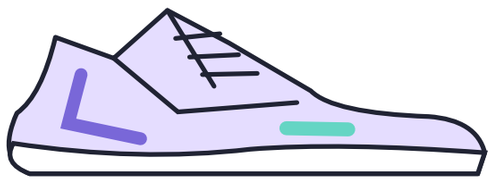
В корзине также есть City Walk × 1.
**Итого: 2 × 7 990 + 5 990 = 21 970 ₽.**

↳ Сервер нужен на каждом шаге, где мы читаем или меняем данные магазина.

# Интерфейс показывает и отправляет

Кнопка связывает действие человека с запросом к серверу.

ЭКРАН ПОКУПАТЕЛЯ · ИЛЛЮСТРАЦИЯ



**North Runner**

7 990 ₽ за пару

42 ~~43~~

Выбрано: 2 пары

**Добавить в корзину**

1. **Показать данные**
   Название, цену и размеры интерфейс получил от API.
2. **Собрать выбор**
   Размер 42 соответствует записи с ID 101. Количество — 2.
3. **Отправить запрос**
   Сервер получит ID размера и количество, затем вернёт результат.

↳ Отключённая кнопка удобна пользователю. Правила покупки всё равно проверяет сервер.

# Сервер проверяет правила магазина

Запрос — просьба выполнить действие. Сервер решает, допустимо ли оно.

Перед оформлением заказа

- ✓ Пользователь вошёл в аккаунт
- ✓ Корзина принадлежит ему
- ✓ Количество больше нуля
- ✓ На складе достаточно пар
- ✓ Сумма рассчитана по ценам из БД

Псевдокод · описывает идею — Учебный пример

```
пользователь = проверить_доступ()
корзина = найти_его_корзину()
проверить_количества(корзина)
проверить_остатки(корзина)
сумма = рассчитать_по_ценам_базы()
оформить_заказ(корзина, сумма)
```

↳ Пользователь сообщает свой выбор. Сервер проверяет его и рассчитывает результат.

# База помнит то, что мы сохранили

Приложение можно перезапустить. Записанный заказ должен остаться.

ПРИЛОЖЕНИЕ

Обрабатывает запросы.
Временные значения живут в памяти.

перезапуск ↻ память процесса заново

**После запуска**

Приложение снова читает сохранённые данные из PostgreSQL.

СОХРАНЁННЫЙ ЗАКАЗ · ФРАГМЕНТ ТАБЛИЦЫ

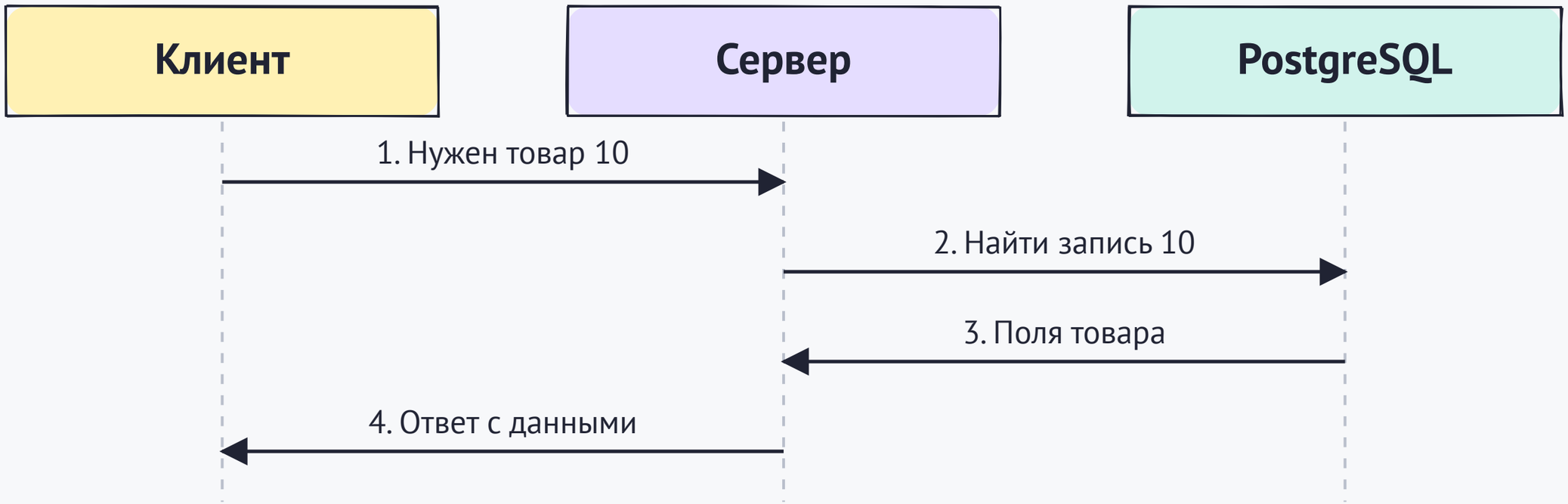
| | |
|---|---|
| id | 501 |
| user_id | 7 · Аня |
| total | 21970.00 |
| status | created |

Каталог, пользователи, корзины и заказы хранятся в своих таблицах.

↳ Память приложения и постоянное хранилище решают разные задачи.

# Клиент, сервер и база

Проследим чтение одной карточки товара.




↳ В этой архитектуре клиент обращается к API, а с базой работает сервер.

01 Что мы собираем

# У каждой технологии своя работа

Пока достаточно понять место инструмента. Код разберём в следующих главах.

### Запустить

**Python + venv**
Код и его окружение

**Docker Compose**
Запуск PostgreSQL

**.env**
Локальные настройки

### Обработать запрос

**FastAPI + Swagger**
Маршруты и проверка API

**Pydantic**
Входные данные и ответы

**passlib + python-jose**
Пароли и токены доступа

### Работать с данными

**PostgreSQL**
Хранение и запросы

**SQLAlchemy**
Работа с БД из Python

**Alembic**
Изменения схемы БД

↳ Инструменты складываются в одну систему: запуск → запрос → правила → данные.

# Разбираем и сразу проверяем

Каждая тема приводит к небольшому действию, которое можно повторить.

01

**Понять**

Какую задачу решаем

02

**Увидеть**

Схема и данные

03

**Прочитать**

Код или псевдокод

04

**Проверить**

Что получилось

05

**Применить**

Шаг практики

**Проверь себя перед второй главой**

Где хранится заказ? Кто проверяет остатки?
Что отправляет клиент после нажатия кнопки?

↳ Следующая глава: читаем запрос и ответ так же уверенно, как сообщение человеку.

# API — способ обратиться к программе

Мы заранее договариваемся, какой запрос отправить и какой ответ получить.

ПРОСЬБА КЛИЕНТА

**«Покажи товары»**

**GET** /products

Адрес и метод определяют операцию API.

JSON · ожидаемый фрагмент ответа — Учебный пример

```
[
  {
    "id": 10,
    "name": "North Runner",
    "price": "7990.00"
  }
]
```

↳ API задаёт понятный программам договор: вход, результат и возможные ошибки.

# Разложим запрос на части

Аня добавляет две пары размера 42 в свою корзину.

HTTP · сокращённая запись запроса — Учебный пример

```
POST /cart/items HTTP/1.1
Host: localhost:8000
Content-Type: application/json
Authorization: Bearer <токен>

{
  "product_size_id": 101,
  "quantity": 2
}
```

1. **Метод: POST**
   Какую операцию просим выполнить.
2. **Путь: /cart/items**
   С какой частью магазина работаем.
3. **Заголовки**
   Адрес сервера, формат данных и доступ.
4. **Тело: JSON**
   Какой размер добавить и сколько пар.

↳ Метод + путь + заголовки + тело. Эти части читаем отдельно.

# Как прочитать адрес API

URL показывает, куда обратиться и какие параметры передать.

```
http://localhost:8000/products?size=42
```

`http://`

Схема

Как обращаемся

`localhost`

Адрес узла

Этот компьютер

`:8000`

Порт

Вход в приложение

`/products`

Путь

Каталог

`?size=42`

Параметры

Фильтр размера

↳ На локальном стенде API и PostgreSQL слушают разные порты.

# Путь выбирает конкретный товар

Число в адресе — идентификатор записи.

**GET** `/products/10`

ТАБЛИЦА PRODUCTS · ФРАГМЕНТ

| id | name | price |
|---|---|---|
| 10 | North Runner | 7990.00 |
| 20 | City Walk | 5990.00 |

**/products/{id}**

Шаблон маршрута. Вместо `{id}` клиент подставляет значение.

`/products/10` → North Runner
`/products/20` → City Walk

↳ ID товара, название товара и размер обуви — разные значения.

# Параметры уточняют выборку

Меняйте фильтры: адрес и ожидаемый результат изменяются вместе.

Бренд **North · id 1** Размер **42** Учебная модель

**GET** `/products?brand_id=1&size=42`

**Ожидаемый ответ:** 200 **· 1 товар**

| Товар | Цена | Размер · остаток |
|---|---|---|
| North Runner | 7 990 ₽ | 42 · 3 пары |

↳ После «?» идут параметры. Символ «&» соединяет несколько условий.

# Тело передаёт данные изменения

Путь выбирает позицию корзины. JSON сообщает новое количество.

HTTP · сокращённая запись запроса — Учебный пример

```
PATCH /cart/items/55
Content-Type: application/json
Authorization: Bearer <токен>

{
  "quantity": 3
}
```

К КАКОЙ ЗАПИСИ

**55** — ID позиции в корзине.
Он не равен ID товара или размера.

ЧТО ИЗМЕНИТЬ

**quantity: 3** — установить три пары.
Было 2 → станет 3.

Сервер также проверит владельца позиции.

↳ Path: какую запись. Body: какие значения. Authorization: кто обращается.

# JSON — данные с понятной структурой

Скобки, имена полей и типы помогают прочитать сообщение.

JSON · сокращённая карточка товара — Учебный пример

```
{
  "id": 10,
  "name": "North Runner",
  "sizes": [42, 43],
  "image_url": null
}
```

1. **{ } — объект**
   Поля с именами: id, name, sizes, image_url.
2. **[ ] — массив**
   Список значений. Здесь — два размера.
3. **"Текст", 10, null**
   Строка, число и отсутствие значения.
4. **true / false — boolean**
   Логическое значение. Например, is_admin в публичном профиле.

↳ Корректный JSON ещё не означает, что данные подходят нашему API.

# Ответ тоже состоит из частей

Сначала узнаём исход запроса, затем читаем данные.

HTTP · сокращённый ожидаемый ответ — Учебный пример

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 10,
  "name": "North Runner",
  "price": "7990.00"
}
```

1. **Статус: 200**
   Запрос выполнен успешно.
2. **Заголовки**
   Content-Type сообщает формат тела.
3. **Тело ответа**
   Поля карточки, которые получил клиент.

↳ Ответ нужно читать целиком: статус объясняет исход, тело — подробности.

02 Как читать запросы и ответы

# Метод говорит, что мы хотим сделать

Один путь может поддерживать несколько разных операций.

| Метод | Пример | Что просим |
|---|---|---|
| **GET** | `/products/10` | Прочитать карточку |
| **POST** | `/products` | Создать товар · администратор |
| **PUT** | `/products/10` | Заменить редактируемые поля карточки |
| **PATCH** | `/cart/items/55` | Изменить указанное поле: quantity |
| **DELETE** | `/cart/items/55` | Удалить позицию из своей корзины |

↳ Для PUT передаём полный набор редактируемых полей. Для PATCH — выбранные изменения.

02 Как читать запросы и ответы

# Адрес называет ресурс магазина

REST помогает организовать API вокруг данных и действий над ними.

| | |
|---|---|
| `/products` | Каталог товаров |
| `/brands` | Список брендов |
| `/cart` | Своя корзина |
| `/orders` | Свои заказы |

**Коллекция**

`GET /products`

Список карточек

**Одна запись**

`GET /products/10`

Карточка с ID 10

↳ Адрес выбирает ресурс, метод — действие, контракт — допустимые данные и результат.

# Успех бывает разным

Статус помогает понять, что именно завершилось.

**200 OK**

## Получено

`GET /products/10`

Карточка найдена.
В теле — её данные.

```
{"id": 10, …}
```

**201 Created**

## Создано

`POST /orders`

Появился заказ.
В теле — результат.

```
{"id": 501, …}
```

**204 No Content**

## Удалено

`DELETE /cart/items/55`

Позиция удалена.
Тела ответа нет.

Пустое тело

↳ Это выбранные ответы нашего API. После 204 не пытаемся читать JSON.

# Отказ тоже должен быть понятным

Одинаковая красная надпись скрывает разные причины.

| | | |
|---|---|---|
| 401 | Нет действительных данных входа | Корзина без токена |
| 403 | Недостаточно прав | Обычный пользователь создаёт товар |
| 404 | Ресурс не найден | Неизвестный ID товара |
| 409 | Конфликт с текущим состоянием | Заказываем 2 пары, осталась 1 |
| 422 | Входные данные не прошли проверку | quantity = 0 при требовании больше 0 |

409 для нехватки остатка — выбранное соглашение магазина. Тело ответа объясняет причину.

↳ Сначала определяем причину отказа, затем решаем, что исправлять.

# Иногда проблема на стороне сервера

Неожиданный сбой программы требует диагностики.

КЛИЕНТ ВИДИТ

**500**

Не удалось выполнить запрос.

**Что выясняем**

Какой запрос отправили?
Что ожидали?
Где возникло исключение?

Python traceback · условный пример — Учебный пример

```
Traceback (most recent call last):
  File "services/catalog.py", line 18
    return product.name
AttributeError:
  'NoneType' has no attribute 'name'
```

↳ Статус сообщает о сбое. Причину ищем в серверной диагностике и воспроизводимом запросе.

# Теперь прочитай запрос сам

Назови действие, ресурс и каждое условие. Затем открой разбор.

**GET** `/products?brand_id=1&size=42&max_price=8000`

**Читаем список товаров.** Бренд — 1, размер — 42, цена — не выше 8 000 ₽. В наших данных подходит North Runner за 7 990 ₽.

Если совпадений нет: **200** и пустой список `[]`. Этот запрос не создаёт заказ и не списывает остаток.

Условия: бренд 1, размер 42, цена до 8 000 включительно. Ответ объясняет выбранный учебный контракт.

---

↳ Дальше научимся читать Python-код, который обрабатывает такие запросы.

# Функция превращает вход в результат

Начнём с цены одной позиции: две пары по 7 990 ₽.

Python · запускаемый пример

```
def line_total(price, quantity):
    return price * quantity

result = line_total(7990, 2)
print(result)
```

price = **7990**

quantity = **2**

7990 × 2

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
15980
```

Результат вернулся в переменную `result`.

↳ def объявляет функцию. Вызов запускает её. return возвращает вычисленное значение.

# Список хранит позиции, словарь — поля

Одна позиция корзины состоит из ID размера и количества.

Python · запускаемый пример

```
cart = [
    {"product_size_id": 101, "quantity": 2},
    {"product_size_id": 201, "quantity": 1},
]
for item in cart:
    print(item["product_size_id"],
item["quantity"])
```

**cart — список из двух словарей**

`for` берёт очередную позицию. `item["quantity"]` читает её поле.

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
101 2
201 1
```

Каждая строка вывода: ID размера, затем количество.

↳ В памяти это объекты Python. Для передачи по HTTP их преобразуют в JSON.

# Аннотация описывает ожидаемый тип

Обычный Python сам не отклоняет аргумент только из-за аннотации.

Python · запускаемый пример

```python
def twice(quantity: int) -> int:
    return quantity * 2

print(twice(2))
print(twice("2"))
```

**quantity: int → ожидаем число**

→ `int` в записи функции также описывает ожидаемый результат.

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
4
22
```

4 — число.
22 — строка: `"2" * 2` повторяет текст.

↳ Аннотации помогают читать и проверять код. Проверку входа API выполняет отдельный механизм.

# Класс описывает объект с данными

Объект хранит своё состояние в атрибутах.

Python · запускаемый пример

```
class Product:
    def __init__(self, name):
        self.name = name

shoe = Product("North Runner")
print(shoe.name)
```

ОБЪЕКТ SHOE В ПАМЯТИ

**Product**

`name` → North Runner

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
North Runner
```

↳ Обычный класс ещё не создаёт таблицу и не проверяет входные данные.

# Импорт соединяет файлы программы

main-файл собирает приложение из небольших частей.

Файлы учебного приложения — Учебный пример

```
sneaker-examples/
├── demo.py
├── schemas.py
├── services.py
└── routers/
    ├── __init__.py
    └── catalog.py
```

Python · фрагмент demo.py

```python
from fastapi import FastAPI
from routers import catalog

app = FastAPI()
app.include_router(catalog.router)
```

`catalog` — импортированный модуль.
`catalog.router` — объект внутри него.

↳ Импорт даёт доступ к определениям. include_router подключает операции к приложению.

# Исключение прерывает обычный путь

Недопустимое количество превращаем в явную ошибку.

Python · запускаемый пример

```
def check_quantity(quantity):
    if quantity <= 0:
        raise ValueError("Нужно больше нуля")
    return quantity

try:
    check_quantity(0)
except ValueError as error:
    print(error)
```

**raise → поиск обработчика**

При quantity = 0 строка `return quantity` уже не выполнится.

✓ ПРОВЕРЕННЫЙ ВЫВОД

Нужно больше нуля

`except ValueError` поймал ошибку и вывел её текст.

↳ Исключение Python и HTTP-ответ — разные вещи. Их связывает обработка ошибки.

# Деньги считаем с Decimal

Десятичную цену создаём из строки, сохраняя исходные цифры.

Python · запускаемый пример

```python
from decimal import Decimal

print(0.1 + 0.2)
print(Decimal("0.1") + Decimal("0.2"))
total = Decimal("7990.00") * 2
print(total)
```

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
0.30000000000000004
0.3
15980.00
```

**Decimal("7990.00")**

Сначала точное значение цены.
Потом умножение на количество.

↳ float хранит двоичное приближение. Для цен используем согласованную десятичную арифметику.

# Пройди по коду самостоятельно

Для каждого вызова проверь условие и найди строку return.

Python · запускаемый пример

```
def can_buy(stock, quantity):
    if quantity <= 0:
        return False
    return quantity <= stock

print(can_buy(3, 2))
print(can_buy(3, 4))
print(can_buy(3, 0))
```

**Проверенный вывод**

```
True
False
False
```

2 ⩽ 3 → True.
4 ⩽ 3 → False.
0 ⩽ 0 → ранний return False.

↳ Сначала подставь значения. Затем следуй только по той ветке, условие которой выполнено.

# Кто запускает FastAPI

Uvicorn принимает сетевое соединение и передаёт запрос приложению.




ИЗ ПАПКИ PRESENTATION/SNEAKER-EXAMPLES · ПОСЛЕ УСТАНОВКИ ЗАВИСИМОСТЕЙ

```
python -m uvicorn minimal:app --port 8000
```

↳ minimal — файл minimal.py. app — объект FastAPI внутри этого файла.

# Минимальное приложение — семь строк

Этот файл можно запустить и проверить настоящим HTTP-запросом.

Python · запускаемый пример

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health():
    return {"status": "ok"}
```

1. **Импортировать FastAPI**
   Получить класс из установленной библиотеки.
2. **Создать app**
   Объект приложения хранит маршруты.
3. **Зарегистрировать /health**
   GET по этому пути вызывает health().
4. **Вернуть данные**
   Словарь станет JSON-ответом клиенту.

↳ Проверенный результат GET /health: 200 и {"status": "ok"}.

# Метод и путь выбирают обработчик

Декоратор связывает HTTP-операцию с Python-функцией.

**GET** `/health`

Python · фрагмент minimal.py

```python
@app.get("/health")
def health():
    return {"status": "ok"}
```

**Совпали метод и путь**

Вызываем `health()`.
Получаем 200.

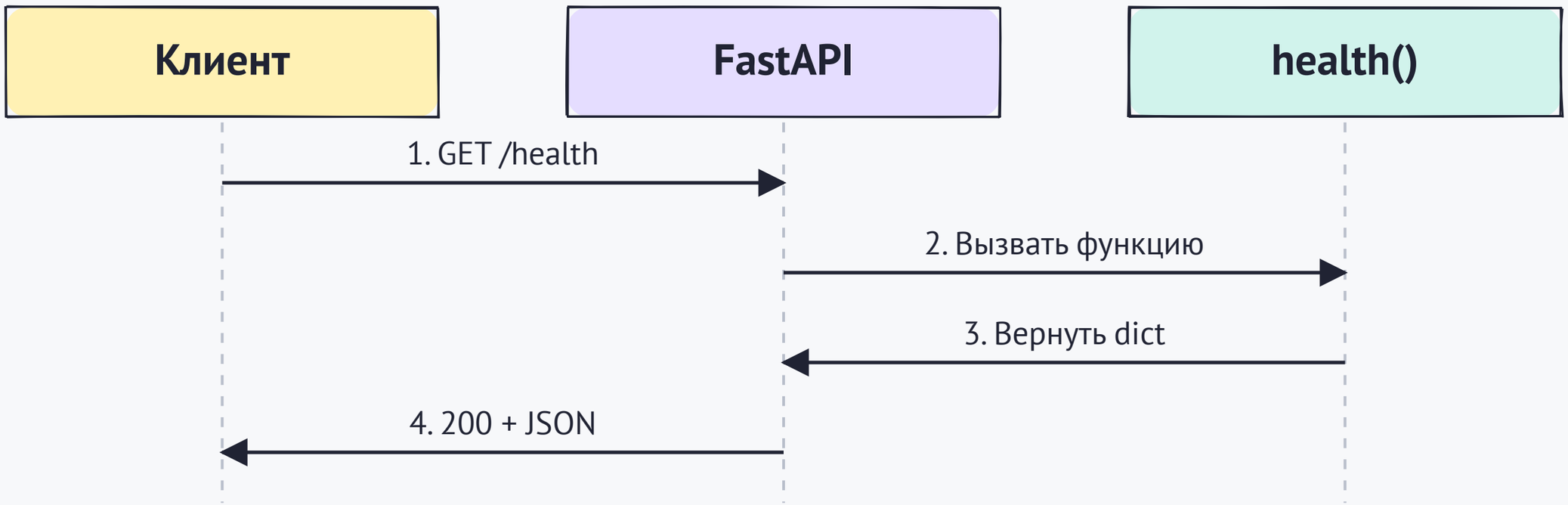
`POST /health` → 405
Путь есть, метод не поддерживается.

`GET /missing` → 404
Такого маршрута в примере нет.

↳ Имя функции — для Python. Внешний адрес задаётся в декораторе.

# Проследим один запрос до ответа

Внутри обработчика работаем с Python-данными. Клиент получает HTTP.




↳ Запрос → выбор функции → return → сериализация → ответ.

# Проверяем запрос через Swagger

Открой /docs у запущенного учебного приложения.

СХЕМА ИНТЕРФЕЙСА /DOCS

`GET /health`

1. Открыть операцию
2. Нажать **Try it out**
3. Нажать **Execute**

---

**200** `{"status": "ok"}`

1. **Проверить адрес**
   http://127.0.0.1:8000/docs
2. **Прочитать Server response**
   Статус и тело после Execute — фактический ответ.
3. **Отличить пример от ответа**
   Example Value показывает образец схемы. Он не доказывает выполнение запроса.

---

↳ Swagger отправляет обычный HTTP-запрос. Это наш клиент для практики.

# Pydantic проверяет данные на входе

Схема задаёт поля, типы и ограничения запроса.

Python · запускаемый пример

```python
from pydantic import BaseModel, Field

class CartItemCreate(BaseModel):
    product_size_id: int = Field(gt=0)
    quantity: int = Field(gt=0, strict=True)
```

**Field(gt=0, strict=True)**

Целое число больше нуля.
Строку «2» и boolean не принимаем.

✓ ПРОВЕРЕННЫЙ ВВОД

```
{"product_size_id": 101, "quantity": 2}
```

Такое тело запроса проходит схему.

↳ Форма данных проверена. Наличие размера и остаток ещё предстоит проверить в базе.

# Неверный ввод не попадает в обработчик

Переключай количество и сравни проверенные ответы FastAPI.

quantity 0 · ноль

JSON · вход /demo/cart/items

```
{
  "product_size_id": 101,
  "quantity": 0
}
```

**422**

```
{
  "loc": ["body","quantity"],
  "type": "greater_than",
  "ctx": {"gt":0}
}
```

Тело обработчика: **не вызвано**

quantity должно быть больше 0.

↳ 422 объясняет, где вход не прошёл схему. Вызов тела обработчика проверяем отдельно.

# Схема ответа выбирает видимые поля

Внутренний объект может содержать больше данных, чем получает клиент.

Python · схема и сокращённый обработчик

```python
class UserRead(BaseModel):
    id: int
    email: str

@app.get("/demo/user",
         response_model=UserRead)
def example_user():
    return internal_user
```

`id` `email` ~~`hashed_password`~~

✓ ПРОВЕРЕННЫЙ JSON-ОТВЕТ

```
{
  "id": 7,
  "email": "anya@example.com"
}
```

↳ response_model=UserRead фильтрует ответ. Пароль и его хэш клиенту не возвращаем.

# На входе и выходе разные данные

Клиент задаёт свойства товара. Сервер добавляет созданный ID.

ЗАПРОС · PRODUCTCREATE

`name` `price`

```
{"name": "North Runner",
 "price": "7990.00"}
```

Название непустое.
Цена — Decimal больше 0.

→

ОТВЕТ · PRODUCTREAD

`id` `name` `price`

```
{"id": 10,
 "name": "North Runner",
 "price": "7990.00"}
```

201 Сервер сформировал ответ.

↳ ProductCreate описывает вход. ProductRead описывает публичный результат.

# Depends передаёт готовое значение

FastAPI вызывает зависимость и подставляет её результат в обработчик.

Python · фрагмент demo.py

```python
from fastapi import Depends, Query
def page_size(
    limit: int = Query(10, ge=1, le=50)
):
    return limit

@app.get("/demo/page")
def page(limit: int = Depends(page_size)):
    return {"limit": limit}
```

1 Запрос `?limit=3`

2 `page_size` проверит и вернёт 3

3 `page` получит `limit=3`

↳ Передаём Depends функцию page_size, а не результат её ручного вызова.

# Роутер собирает связанные операции

Каталог получает свой файл и общий префикс адресов.

Python · сокращённый routers/catalog.py

```python
from fastapi import APIRouter
from services import list_products

router = APIRouter(prefix="/products")

@router.get("")
def catalog():
    return list_products()
```

`/products` + `""` = `/products`

**В demo.py**

```
app.include_router(catalog.router)
```

Позже отдельные роутеры:
auth · cart · orders

↳ APIRouter группирует маршруты. include_router делает их частью приложения.

# Роутер принимает запрос, сервис решает задачу

Для оформления заказа нам понадобятся разные уровни ответственности.

**Роутер**

Принять HTTP-запрос.
Вызвать сервис и вернуть ответ.

Псевдокод · будущий POST /orders — Учебный пример

```
пользователь = проверить_доступ()
заказ = сервис.оформить(пользователь)
вернуть_ответ(заказ)
```

**Сервис заказа**

- ✓ Прочитать свою корзину
- ✓ Проверить остатки
- ✓ Рассчитать сумму
- ✓ Сохранить заказ и списать склад
- ✓ Очистить корзину

Изменения объединит транзакция.

↳ Дальше подключим PostgreSQL и разберём, как эти действия меняют реальные данные.

05 Окружение и Docker

# Из чего состоит локальный запуск

Python работает на компьютере. PostgreSQL — в контейнере Docker.




↳ Два процесса общаются по сети. У каждого свои зависимости, настройки и журнал ошибок.

# venv изолирует библиотеки проекта

Одна папка проекта — своё окружение и свой набор зависимостей.

Терминал · macOS / Linux — Учебный пример

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip --version
python -c "import sys;
print(sys.executable)"
```

**До активации**

`python` выбирается из системного PATH.

**После активации**

`python` указывает на `.venv/bin/python`.

↳ python -m pip устанавливает пакеты для того Python, которым вы запускаете приложение.

# requirements фиксирует зависимости

Сохраняем проверенные версии, чтобы повторить окружение.

Фрагмент проверенного requirements.txt — Учебный пример

```
fastapi==0.141.1
pydantic==2.13.5
uvicorn==0.52.4
httpx==0.28.1
```

Установка из файла — Учебный пример

```
python -m pip install -r
requirements.txt
python -m pip check
```

**Для полного воспроизведения**

Фиксируем также транзитивные пакеты и версию Python.

↳ Файл зависимостей — рецепт установки. Его совместимость подтверждается запуском и проверками.

# Образ — шаблон для контейнера

postgres:17 содержит подготовленное окружение PostgreSQL.

ОБРАЗ POSTGRES:17

- PostgreSQL
- Системные библиотеки
- Инструкция запуска

Скачать образ — Учебный пример

```
docker pull postgres:17
```

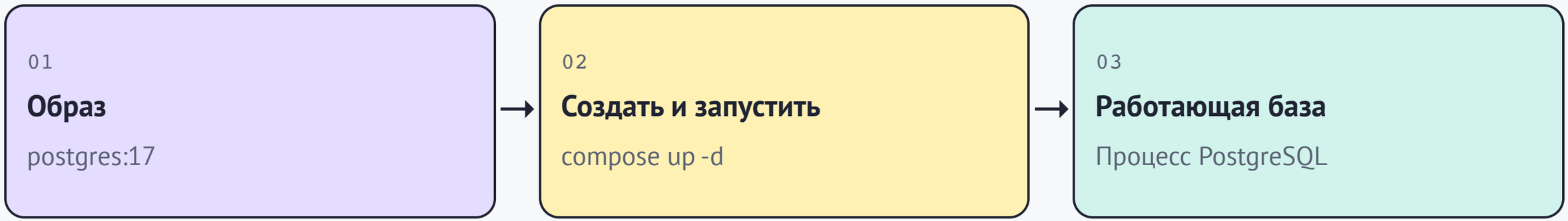
**Образ не равен данным магазина**

Из одного образа можно запустить несколько отдельных контейнеров.

↳ Образ содержит файлы и настройки запуска. Работающий процесс появится в контейнере.

# Контейнер — запущенный экземпляр

У контейнера есть состояние, сеть и журнал процесса.




Команды из папки с compose.yaml — Учебный пример

```
docker compose ps
docker compose logs --tail 30 db
docker compose stop db
```

↳ stop останавливает процесс. Следующий up -d снова поднимает сервис.

# Compose описывает запуск в файле

Один сервис db: образ, переменные, порт и место хранения.

compose.yaml · фрагмент — Учебный пример

```
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "127.0.0.1:55432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
```

### Отступы задают структуру

**db** — имя сервиса.
**pgdata** — именованный том.

Запуск полного файла из примеров — Учебный пример

```
docker compose config --quiet
docker compose up -d --wait
```

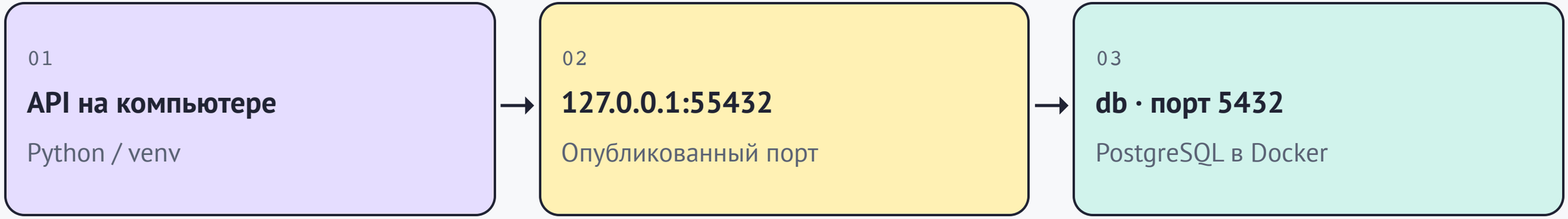
↳ Полный файл рядом с презентацией также объявляет pgdata, пользователя, базу и healthcheck.

05 Окружение и Docker

# Адрес зависит от места запуска API

localhost всегда означает текущее сетевое окружение.

API запущен: На компьютере — наш стенд




Публикация порта: `127.0.0.1:55432:5432` · компьютер → контейнер

---

↳ С компьютера: 127.0.0.1:55432. Между сервисами одной сети Compose: db:5432.

# Том переживает замену контейнера

Файлы PostgreSQL находятся в именованном томе pgdata.

**Контейнер A**

PostgreSQL пишет файлы базы

**Контейнер B**

После пересоздания читает те же файлы

↔

**Том pgdata**

Каталог данных PostgreSQL
**таблицы · записи · индексы**

В PostgreSQL 17 монтируем том в `/var/lib/postgresql/data`.

---

↳ Пересоздание контейнера и удаление тома — разные операции. Том не заменяет резервную копию.

# DATABASE_URL — адрес подключения

Одна строка отвечает на вопросы: чем, кем, куда и в какую базу.

Учебный адрес · пароль вымышленный — Учебный пример

```
postgresql://lesson:lesson-only@127.0.0.1:55432/sneaker_lesson
```

`postgresql`
Протокол

`lesson`
Пользователь

`lesson-only`
Пароль

`127.0.0.1:55432`
Хост и порт

`sneaker_lesson`
Имя базы

↳ Один хост может обслуживать несколько баз. Имя базы — не имя таблицы и не имя контейнера.

# .env хранит настройки окружения

Пример файла объясняет формат. Реальный файл подставляет значения.

.env.example · учебные значения — Учебный пример

```
POSTGRES_USER=lesson
POSTGRES_PASSWORD=lesson-only
POSTGRES_DB=sneaker_lesson
DB_PORT=55432
```

Сначала скопировать — Учебный пример

```
cp .env.example .env
```

### Compose

Читает .env для подстановки `${...}` в YAML.

### Python

Нужна явная загрузка настроек: например, `load_dotenv()` перед `os.environ["DATABASE_URL"]`.

↳ .env исключаем из Git. .env.example коммитим без реальных секретов.

# API запущен — а база доступна?

Проверка готовности должна действительно обратиться к PostgreSQL.

Псевдокод GET /health — Учебный пример

```
попробовать:
    открыть соединение с таймаутом
    выполнить SQL: SELECT 1
    закрыть соединение
    вернуть 200, {status: "ok"}
если база недоступна:
    вернуть 503, {status: "unavailable"}
```

**GET /live → 200**

Процесс API отвечает. Базу не проверяем.

**GET /health → 200**

Наш readiness-маршрут проверил соединение и простой SQL-запрос.

↳ SELECT 1 проверяет соединение и выполнение SQL. Наличие таблиц и миграций проверяем отдельно.

# Если не запускается: идём по цепочке

Проверяем ближайшую причину, затем повторяем тот же запрос.

**1. Docker недоступен** — `docker info`

Запустить Docker Desktop / engine

**2. Контейнер не healthy** — `docker compose ps`

Прочитать logs db; проверить порт

**3. Нет соединения** — Хост → порт → база

Сверить адрес с местом запуска API

**4. Отказ в доступе** — Роль и пароль

Проверить учётные данные уже созданной БД

**5. Модуль не найден** — `python -m pip --version`

Проверить venv и requirements

↳ Ошибка — подсказка, на каком участке оборвалась цепочка. Исправляем одну причину за раз.

# Почему списка Python недостаточно

Каталог должен пережить перезапуск и быть общим для всех покупателей.

Память одного процесса — Учебный пример

```
products = [
    {"id": 10, "stock": 3}
]
# После перезапуска создаётся заново
```

**Несколько процессов**

У каждого — своя память и своя копия списка.

**PostgreSQL**

Хранит данные на диске и обслуживает разные соединения.

**Правила согласованности**

Ограничения и транзакции помогают сохранять корректные данные.

↳ База хранит общее состояние. Без корректных запросов и транзакций она не решает гонки автоматически.

# Таблица: строки и столбцы

Строка описывает один товар. Столбец хранит один вид значения.

PRODUCTS

| id | name | brand_id | price |
|---|---|---|---|
| **10** | North Runner | 1 | 7990 |
| **20** | City Walk | 2 | 5990 |
| **30** | North Trail | 1 | 9990 |

3 строки → 3 товара

price → цена каждой строки

✓ Выполнено в PostgreSQL

↳ Таблица задаёт общую структуру. Каждая запись получает свои значения полей.

# Тип поля объясняет его значение

Цена, количество, текст и время требуют разных правил хранения.

`text`

**name**

North Runner

Текст

`numeric(10,2)`

**price**

7990.00

Точное десятичное число

`integer`

**stock**

3

Целое количество

`boolean`

**is_admin**

false

Да / нет

`timestamptz`

**created_at**

2026-10-01 12:00+00

Момент времени

`NULL`

**image_url**

нет значения

Не пустая строка и не 0

↳ Тип ограничивает форму значения. CHECK и NOT NULL добавляют правила к этому типу.

# Первичный ключ отличает записи

Название может повториться. ID должен однозначно найти нужную строку.

ОТДЕЛЬНЫЙ МИНИ-ПРИМЕР · ОДИНАКОВЫЕ НАЗВАНИЯ

| id | name |
|---|---|
| **10** | Runner |
| **20** | Runner |

PRIMARY KEY · первичный ключ — Учебный пример

```
id integer GENERATED BY DEFAULT
AS IDENTITY PRIMARY KEY
```

**Найти одну конкретную запись**

```
WHERE id = 20
```

Название для этого ненадёжно.

↳ PRIMARY KEY означает уникальность и запрет NULL. ID не обязан идти без пропусков.

# Семь таблиц — семь задач магазина

Разделяем сведения, которые меняются и используются по-разному.

**`users`**
Кто покупает

**`brands`**
Кто выпускает

**`products`**
Что продаём

**`product_sizes`**
Размер и остаток

**`cart_items`**
Что выбрано сейчас

**`orders`**
Факт покупки

**`order_items`**
Состав и цена покупки

↳ Связи соединяют таблицы обратно: от позиции корзины можно дойти до размера, товара и бренда.

# Пользователи и бренды: первые записи

ID нужен для связей. Остальные поля описывают сущность.

USERS

| id | email | is_admin |
|---|---|---|
| **7** | anya@example.com | false |
| **8** | boris@example.com | false |

BRANDS

| id | name |
|---|---|
| **1** | North |
| **2** | City |

↳ UNIQUE(email) запрещает повтор значения email. UNIQUE(name) для бренда — решение нашего примера.

# Товар и размер — разные сущности

Цена относится к карточке. Остаток относится к её размеру.

PRODUCTS

| id | name | price |
|---|---|---|
| **10** | North Runner | 7990 |

PRODUCT_SIZES

| id | product_id | size | stock |
|---|---|---|---|
| 101 | **10** | 42 | 3 |
| 102 | **10** | 43 | 0 |

↳ North Runner существует один раз. Его размеры 42 и 43 имеют независимые остатки: 3 и 0.

# Внешний ключ связывает две таблицы

products.brand_id ссылается на существующий brands.id.

BRANDS

| id | name |
|---|---|
| **1** | North |
| **2** | City |

PRODUCTS

| id | name | brand_id |
|---|---|---|
| 10 | North Runner | **1** |
| 20 | City Walk | **2** |
| 30 | North Trail | **1** |

↳ brand_id = 1 → brands.id = 1 → North. Название бренда берём из связанной строки.

# Один ко многим: где хранится ссылка

Внешний ключ находится на стороне «много».

**brands · 1**

North · id=1

1 → N

**products · много**

10 · North Runner

30 · North Trail

**products · 1**

North Runner · id=10

1 → N

**product_sizes · много**

101 · размер 42

102 · размер 43

↳ У многих товаров один brand_id. У многих размеров один product_id.

# 10, 42 и 101 — три разных значения

Чтобы положить размер в корзину, нужен ID записи размера.

01
**product_id = 10**
Товар North Runner

→

02
**size = 42**
Номер размера обуви

→

03
**id = 101**
Запись product_sizes

Тело запроса в корзину — Учебный пример

```
{"product_size_id": 101, "quantity": 2}
```

↳ В API передаём product_size_id=101. Число 42 само по себе не определяет товар.

# Корзина хранит выбор пользователя

Одна строка: кто выбрал, какой размер и сколько пар.

CART_ITEMS

| id | user_id | product_size_id | quantity |
|---|---|---|---|
| 55 | **7** | **101** | 2 |
| 56 | **7** | **201** | 1 |
| 57 | **8** | **102** | 1 |

Пользователь 7 → позиции 55 и 56

Пользователь 8 → позиция 57

↳ Связь user_id хранит владельца. Проверку доступа к его строкам должен выполнять сервер.

# Заказ сохраняет факт покупки

Шапка описывает покупку целиком. Позиции фиксируют её состав и цены.

**orders · id=501**

user_id: **7** · completed
total: **21 970 ₽**

**Сумма при оформлении**

2 × 7 990 + 1 × 5 990
= **21 970 ₽**

ORDER_ITEMS · ORDER_ID=501

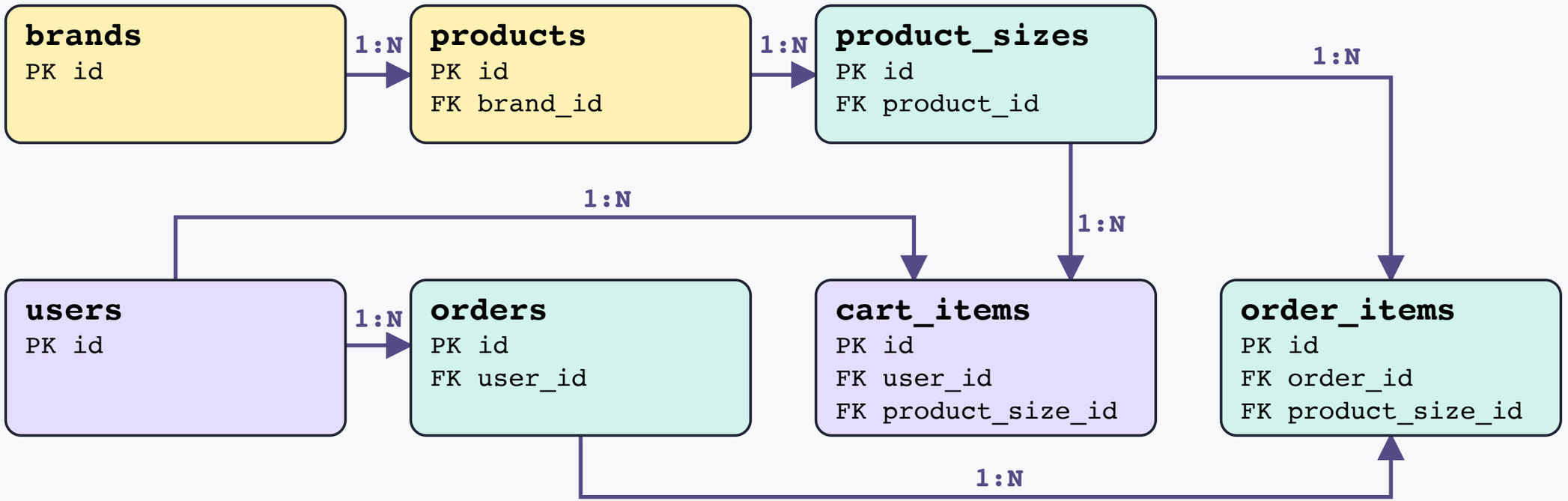
| id | product_size_id | quantity | price_at_purchase |
|---|---|---|---|
| 601 | **101** | 2 | 7990 |
| 602 | **201** | 1 | 5990 |

↳ price_at_purchase — цена одной пары на момент покупки. История не пересчитывается по текущему каталогу.

# Вся схема магазина на одном экране

PK — первичный ключ. FK — внешний ключ. Стрелка: родитель → дочерние строки.

Подсветить связи: Вся схема 1:N — один ко многим




↳ Каждая стрелка — связь по ID. У родителя может быть 0…N дочерних строк.

# Ограничения защищают саму базу

Проверки работают и при записи через API, и при прямом SQL.

**NOT NULL** `name = NULL`

Обязательное значение отсутствует

✓ Отклонено · 23502

**UNIQUE** `Второй бренд North`

Значение уже существует

✓ Отклонено · 23505

**FOREIGN KEY** `brand_id = 999`

Нет такого бренда

✓ Отклонено · 23503

**CHECK** `stock = -1`

Нарушено правило stock ⩾ 0

✓ Отклонено · 23514

Pydantic проверяет вход API. Ограничения PostgreSQL защищают данные независимо от способа записи.

06 PostgreSQL: таблицы и связи

# SELECT читает нужные строки

Выбираем поля, фильтруем, сортируем и ограничиваем результат.

Выборка: Цена до 8 000

✓ Выполнено в PostgreSQL

SQL Учебный пример

```
SELECT id, name, price
FROM products
WHERE price <= 8000
ORDER BY price, id
LIMIT 10;
```

РЕЗУЛЬТАТ

| id | name | price |
|---|---|---|
| 20 | City Walk | 5990 |
| 10 | North Runner | 7990 |

↳ WHERE отбирает строки. ORDER BY задаёт порядок. LIMIT ограничивает количество строк.

# JOIN соединяет строки по ключам

Добавим название бренда к каждому товару.

SQL · INNER JOIN — Учебный пример

```
SELECT p.id, p.name, b.name AS brand
FROM products AS p
JOIN brands AS b ON b.id = p.brand_id
ORDER BY p.id;
```

РЕЗУЛЬТАТ JOIN

| id | name | brand |
|---|---|---|
| 10 | North Runner | **North** |
| 20 | City Walk | **City** |
| 30 | North Trail | **North** |

↳ ON b.id = p.brand_id описывает совпадение ключей. p и b — короткие имена таблиц.

# INSERT, UPDATE, DELETE меняют строки

Отдельный учебный бренд 99: добавили → переименовали → удалили.

1 · INSERT — Учебный пример

```
INSERT INTO brands(id, name)
VALUES (99, 'Demo');
```

ПОСЛЕ ШАГА

| id | name |
|---|---|
| 99 | Demo |

2 · UPDATE — Учебный пример

```
UPDATE brands
SET name = 'Demo Updated'
WHERE id = 99;
```

ПОСЛЕ ШАГА

| id | name |
|---|---|
| 99 | Demo Updated |

3 · DELETE — Учебный пример

```
DELETE FROM brands
WHERE id = 99;
```

ПОСЛЕ ШАГА

| id | name |
|---|---|

0 строк • [ ]

Показываем строку `WHERE id = 99` после выбранного шага.

↳ WHERE определяет, какие строки менять. Без него UPDATE и DELETE затронут все строки таблицы.

# Практика: пройдите по связям

Позиция заказа 601. Найдите покупателя, товар, размер и бренд.

**Кто:** 601 → order 501 → user 7 → anya@example.com

**Что:** 601 → size 101 → product 10 → North Runner, размер 42

**Бренд:** product 10 → brand 1 → North

Цена покупки останется **7 990 ₽**. Текущая цена каталога её не заменяет.

---

↳ Готовы к следующему блоку: представить эти же таблицы и связи в Python через SQLAlchemy.